

Java Inheritance and Implementation

Sample Programs and UML Diagrams for All 8 Cases

Case	Declaration	Status
1	C1 extends C2	Valid
2	C1 extends C2, C3	Invalid
3	C1 implements I1	Valid
4	C1 implements I1, I2	Valid
5	C1 implements C2 extends I1	Invalid
6	I1 extends I2	Valid
7	I1 implements C1	Invalid
8	I1 extends I2, I3	Valid

Important Java rule: A class can extend only one class, but it can implement multiple interfaces. An interface can extend one or more interfaces.

Quick Reference of Rules

The following table summarizes the eight declarations before the detailed programs and UML diagrams.

Syntax	Valid?	Reason
class C1 extends C2	Yes	A class may extend one class.
class C1 extends C2, C3	No	Java does not support multiple class inheritance.
class C1 implements I1	Yes	A class may implement an interface.
class C1 implements I1, I2	Yes	A class may implement multiple interfaces.
class C1 implements C2 extends I1	No	implements requires interfaces and extends must come before implements.
interface I1 extends I2	Yes	An interface may extend another interface.
interface I1 implements C1	No	An interface cannot implement a class.
interface I1 extends I2, I3	Yes	An interface may extend multiple interfaces.

Case 1: C1 extends C2

Status: VALID

Rule / Explanation

A class can extend exactly one class. C1 becomes a subclass of C2 and inherits accessible members from C2.

Sample Java Program

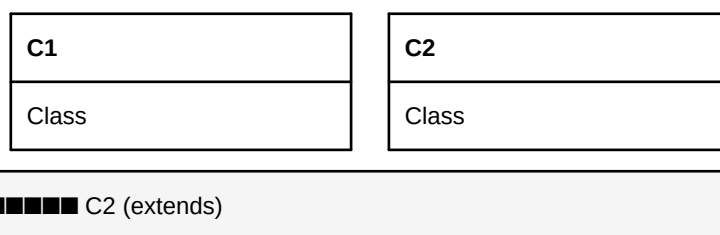
```
class C2 {  
    void display() {  
        System.out.println("This is C2");  
    }  
}  
  
class C1 extends C2 {  
    void show() {  
        System.out.println("This is C1");  
    }  
}  
  
public class Case1 {  
    public static void main(String[] args) {  
        C1 obj = new C1();  
        obj.display();  
        obj.show();  
    }  
}
```

Expected Output

```
This is C2  
This is C1
```

UML Diagram

UML Diagram - Single Class Inheritance



Case 2: C1 extends C2, C3

Status: INVALID

Rule / Explanation

Invalid in Java. A class cannot extend two or more classes. Java does not support multiple inheritance of classes because it can create ambiguity when parent classes contain the same members.

Sample Java Program

```
class C2 {
    void display() {
        System.out.println("This is C2");
    }
}

class C3 {
    void display() {
        System.out.println("This is C3");
    }
}

class C1 extends C2, C3 {
    void show() {
        System.out.println("This is C1");
    }
}

public class Case2 {
    public static void main(String[] args) {
        System.out.println("C1 extends C2, C3 is invalid in Java.");
    }
}
```

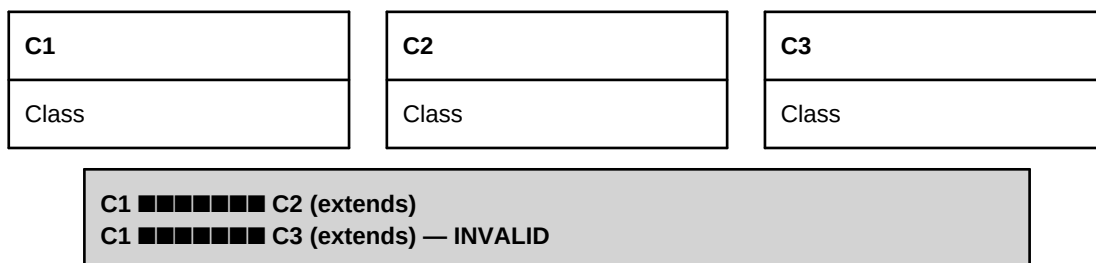
Note: The class declaration shown above is intentionally invalid and will produce a compiler error. The main method is a separate runnable demonstration of the rule.

Expected Output

The declaration class C1 extends C2, C3 is not valid Java syntax.
A class can extend only one class.

UML Diagram

UML Diagram - Multiple Class Inheritance (Invalid)



Case 3: C1 implements I1

Status: VALID

Rule / Explanation

A class can implement one interface. The class must provide implementations for all abstract methods declared by the interface, unless the class itself is abstract.

Sample Java Program

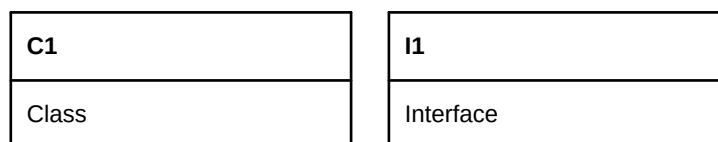
```
interface I1 {  
    void display();  
}  
  
class C1 implements I1 {  
    public void display() {  
        System.out.println("I1 method implemented by C1");  
    }  
}  
  
public class Case3 {  
    public static void main(String[] args) {  
        C1 obj = new C1();  
        obj.display();  
    }  
}
```

Expected Output

I1 method implemented by C1

UML Diagram

UML Diagram - Class Implementing One Interface



C1 -.-> I1 (implements)

Case 4: C1 implements I1, I2

Status: VALID

Rule / Explanation

A class can implement multiple interfaces. It must implement the required abstract methods from all implemented interfaces, provided there are no unresolved method conflicts.

Sample Java Program

```
interface I1 {
    void method1();
}

interface I2 {
    void method2();
}

class C1 implements I1, I2 {
    public void method1() {
        System.out.println("Method from I1");
    }

    public void method2() {
        System.out.println("Method from I2");
    }
}

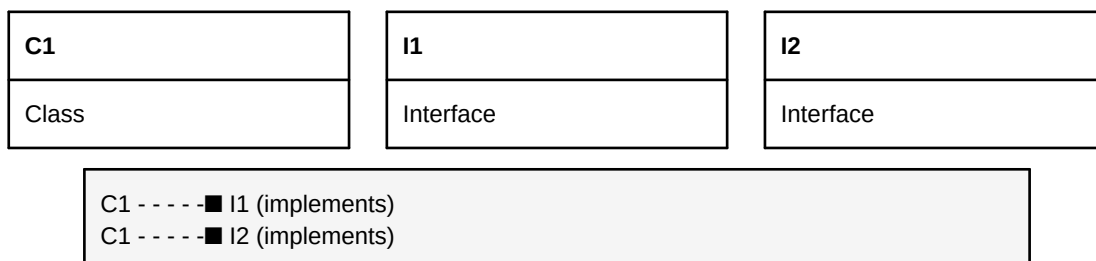
public class Case4 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.method1();
        obj.method2();
    }
}
```

Expected Output

```
Method from I1
Method from I2
```

UML Diagram

UML Diagram - Multiple Interface Implementation



Case 5: C1 implements C2 extends I1

Status: INVALID

Rule / Explanation

Invalid declaration. A class cannot implement another class because implements is used only for interfaces. Also, a class declaration cannot use both extends and implements in the order shown. The valid form, when C2 is a class and I1 is an interface, is: class C1 extends C2 implements I1.

Sample Java Program

```
interface I1 {
    void display();
}

class C2 {
    void show() {
        System.out.println("This is C2");
    }
}

class C1 implements C2 extends I1 {
    public void display() {
        System.out.println("I1 method");
    }
}

public class Case5 {
    public static void main(String[] args) {
        System.out.println("C1 implements C2 extends I1 is invalid in Java.");
        System.out.println("Use: class C1 extends C2 implements I1");
    }
}
```

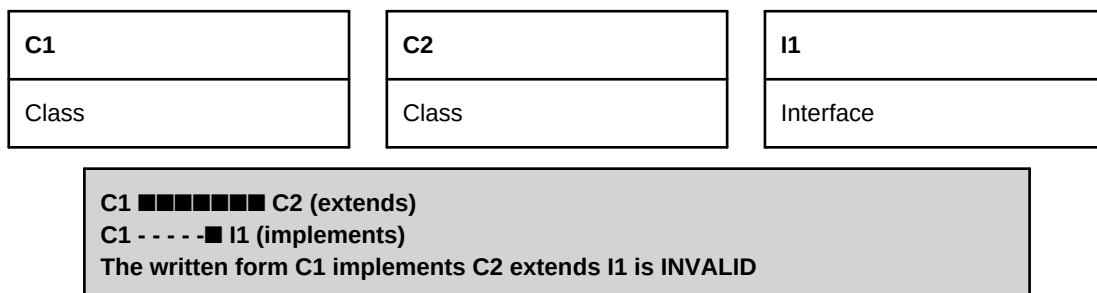
Note: For reference, the legal Java form is shown separately below. The runnable demonstration prints the rule.

Expected Output

```
The declaration is invalid.
implements must be followed by an interface.
The correct combined form is:
class C1 extends C2 implements I1
```

UML Diagram

UML Diagram - Invalid Combined Declaration



Correct legal form for Case 5: class C1 extends C2 implements I1

Case 6: I1 extends I2

Status: VALID

Rule / Explanation

An interface can extend another interface. I1 inherits the abstract/default/static members available through I2. A class implementing I1 must satisfy the inherited interface contract as well.

Sample Java Program

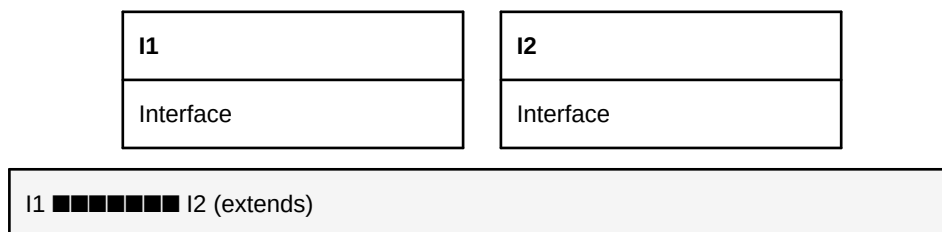
```
interface I2 {  
    void method2();  
}  
  
interface I1 extends I2 {  
    void method1();  
}  
  
class C1 implements I1 {  
    public void method1() {  
        System.out.println("Method from I1");  
    }  
  
    public void method2() {  
        System.out.println("Method from I2");  
    }  
}  
  
public class Case6 {  
    public static void main(String[] args) {  
        C1 obj = new C1();  
        obj.method1();  
        obj.method2();  
    }  
}
```

Expected Output

```
Method from I1  
Method from I2
```

UML Diagram

UML Diagram - Interface Inheritance



Case 7: I1 implements C1

Status: INVALID

Rule / Explanation

Invalid in Java. An interface does not implement a class. An interface extends one or more interfaces. A class implements one or more interfaces.

Sample Java Program

```
class C1 {
    void display() {
        System.out.println("This is C1");
    }
}

interface I1 implements C1 {
    void show();
}

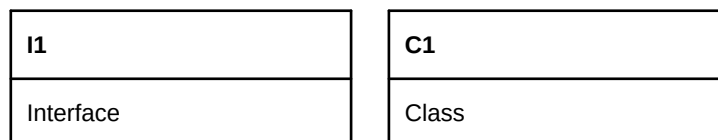
public class Case7 {
    public static void main(String[] args) {
        System.out.println("I1 implements C1 is invalid in Java.");
        System.out.println("An interface can extend interfaces, not implement classes.");
    }
}
```

Expected Output

```
I1 implements C1 is invalid in Java.
An interface can extend interfaces, not implement classes.
```

UML Diagram

UML Diagram - Interface Implementing Class (Invalid)



I1 - - - - ■ C1 (implements) — INVALID
Valid relationship: C1 - - - - ■ I1 (class implements interface)

Case 8: I1 extends I2, I3

Status: VALID

Rule / Explanation

An interface can extend multiple interfaces. This is how Java supports multiple inheritance of interface contracts while avoiding multiple inheritance of classes.

Sample Java Program

```
interface I2 {
    void method2();
}

interface I3 {
    void method3();
}

interface I1 extends I2, I3 {
    void method1();
}

class C1 implements I1 {
    public void method1() {
        System.out.println("Method from I1");
    }

    public void method2() {
        System.out.println("Method from I2");
    }

    public void method3() {
        System.out.println("Method from I3");
    }
}

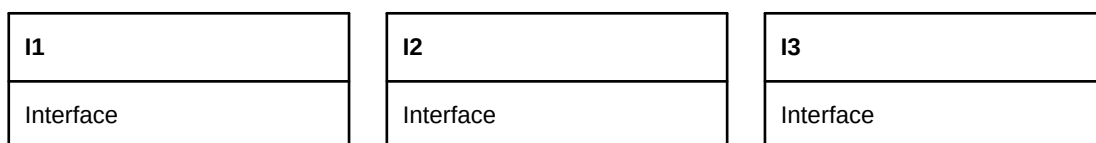
public class Case8 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.method1();
        obj.method2();
        obj.method3();
    }
}
```

Expected Output

```
Method from I1
Method from I2
Method from I3
```

UML Diagram

UML Diagram - Multiple Interface Inheritance



I1 ■■■■■■ I2 (extends)
I1 ■■■■■■ I3 (extends)

Execution Steps

1. Open Eclipse IDE or any Java IDE.
2. Create a Java project named **InheritanceDemo**.
3. For each valid case, create a Java file using the class name shown in the program.
4. For invalid cases, keep the intentionally invalid declaration only for studying the compiler rule; run the separate demonstration program.
5. Run the program and compare the console output with the expected output in this PDF.
6. Use the UML diagrams to understand the relationship between classes and interfaces.
7. Capture console screenshots if your assignment requires them.

Conclusion

Java supports single inheritance between classes and multiple inheritance through interfaces. The `extends` keyword is used for class inheritance and interface inheritance, while `implements` is used by classes to implement interfaces. The eight cases above demonstrate the valid and invalid combinations.